



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения

**ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №8
по дисциплине
«Технологии виртуализации клиент-серверных приложений»**

Выполнил студент группы ИКБО-16-22

Грибков А.С.

Принял преподаватель кафедры ИиППО

Волков М.Ю.

Практические работы выполнены

«__»_____2025 г.

«Зачтено»

«__»_____2025 г.

Москва
2025

СОДЕРЖАНИЕ

1 Цель работы	3
2 Ход работы	4
2.1 Архитектура системы.....	4
2.2 Развертывание инфраструктуры	5
2.2.1 PostgreSQL и Redis	5
2.2.2 Kafka	6
2.2.3 Graylog	6
2.2.4 Prometheus и Grafana	7
2.2.5 Jaeger	7
2.3 Развертывание микросервисов	8
2.3.1 Auth Service	8
2.3.2 Task Service	8
2.3.3 Notification Service	8
2.3.4 KrakenD API Gateway	9
2.4 Конфигурация Helm Charts	9
3 Проверка работоспособности системы	11
3.1 Авторизация и работа с токенами.....	11
3.2 Создание и управление задачами.....	11
3.3 Проверка Kafka	12
3.4 Проверка Graylog	13
3.5 Проверка Prometheus и Grafana	14
3.6 Проверка Jaeger.....	15
3.7 Проверка Persistent Volume.....	16
4 Вывод	18
5 Ответы на вопросы	19
6 Список использованной литературы	22

1 ЦЕЛЬ РАБОТЫ

Разработать Backend систему с использованием Spring Boot, развернуть её в Minikube с помощью Helm-чартов и настроить полный стек наблюдаемости.

Задачи работы:

1. Реализовать систему из минимум 2 сервисов с авторизацией через JWT токены в Redis
2. Настроить взаимодействие между сервисами через Kafka
3. Развернуть KrakenD как API-шлюз
4. Настроить логирование через Graylog, метрики через Prometheus + Grafana, трассировку через Jaeger
5. Обеспечить хранение данных в PostgreSQL с Persistent Volume
6. Настроить HPA, Liveness/Readiness Probes и лимиты ресурсов

2 ХОД РАБОТЫ

2.1 Архитектура системы

Реализована система управления задачами (Task Management System) со следующими компонентами:

- Auth Service (порт 8081) — регистрация, авторизация, хранение токенов в Redis
- Task Service (порт 8082) — CRUD операции с задачами
- Notification Service (порт 8083) — обработка событий из Kafka, управление уведомлениями
- KrakenD (порт 8080) — API Gateway
- PostgreSQL — 3 инстанса (по одной БД на сервис)
- Redis — хранение JWT токенов
- Kafka — шина сообщений
- Graylog — централизованное логирование
- Prometheus + Grafana — метрики
- Jaeger — распределенная трассировка

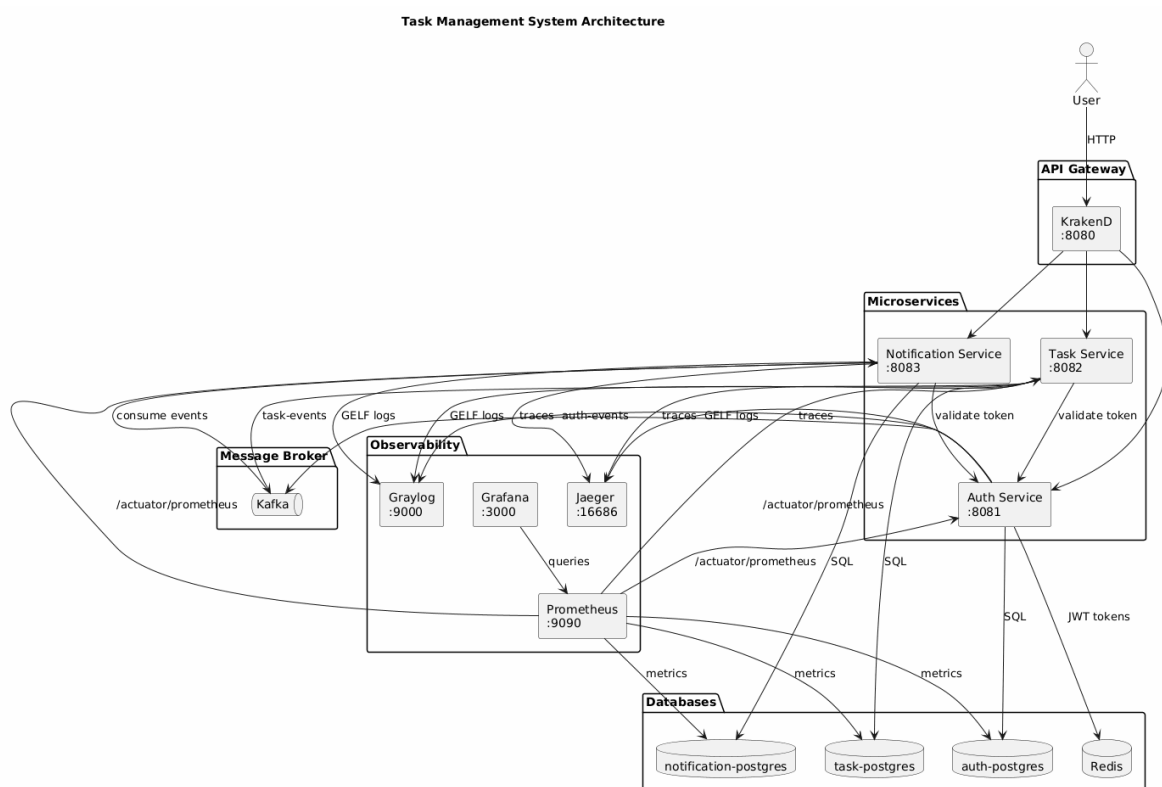


Рисунок 2.1 — Архитектура системы Task Management

2.2 Развертывание инфраструктуры

2.2.1 PostgreSQL и Redis

Для каждого сервиса создана отдельная база данных PostgreSQL с Persistent Volume. Redis используется для хранения JWT токенов с TTL 24 часа.

Листинг 2.1 — Конфигурация PostgreSQL для Auth Service (values.yaml)

```

auth:
  enabled: true
  postgresqlUsername: auth_user
  postgresqlPassword: auth_password
  postgresqlDatabase: auth_db
  persistence:
    enabled: true
    size: 1Gi
  primary:
    resources:
      requests:
        cpu: 250m
        memory: 256Mi
      limits:
        cpu: 500m
        memory: 512Mi
  extraEnvVars:
    - name: POSTGRES_INITDB_ARGS
      value: "-encoding=UTF-8"
  metrics:
    enabled: true
  serviceMonitor:
    enabled: true
  
```

```
alexgribkov@i113971732 docs % kubectl get pods -n task-management | grep po
stgres
auth-postgres-postgresql-0          1/1      Running    0
  93m
notification-postgres-postgresql-0  1/1      Running    0
  91m
task-postgres-postgresql-0          1/1      Running    0
  92m
```

Рисунок 2.2 — Запущенные поды PostgreSQL

2.2.2 Kafka

Kafka развернут в режиме KRaft (без Zookeeper) с двумя топиками: auth-events и task-events.

Листинг 2.2 — Конфигурация Kafka StatefulSet

```
apiVersion: apps/v1 kind: StatefulSet metadata: name: kafka
namespace: task-management spec: serviceName: kafka replicas: 1
template: spec: containers: - name: kafka image: confluentinc/cp-
kafka:7.5.0 env: - name: KAFKA_PROCESS_ROLES value:
"broker,controller" - name: KAFKA_LISTENERS value:
"PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093" - name:
KAFKA_AUTO_CREATE_TOPICS_ENABLE value: "true" resources: requests:
cpu: 200m memory: 512Mi limits: cpu: 500m memory: 1Gi
volumeMounts: - name: data mountPath: /var/lib/kafka/data
volumeClaimTemplates: - metadata: name: data spec: accessModes:
["ReadWriteOnce"] resources: requests: storage: 2Gi
```

2.2.3 Graylog

Graylog развернут совместно с MongoDB и OpenSearch для централизованного сбора логов. Все сервисы отправляют логи через GELF UDP на порт 12201.

Листинг 2.3 — Конфигурация Graylog Deployment

```
apiVersion: apps/v1 kind: Deployment metadata: name: graylog
namespace: task-management spec: template: spec: containers: -
name: graylog image: graylog/graylog:5.2 env: - name:
GRAYLOG_HTTP_EXTERNAL_URI value: "http://localhost:9000/" - name:
```

```
GRAYLOG_ELASTICSEARCH_HOSTS value: "http://opensearch:9200" -
name: GRAYLOG_MONGODB_URI value:
"mongodb://root:graylog_password@mongodb:27017/graylog" ports: -
containerPort: 9000 name: http - containerPort: 12201 name: gelf-
udp protocol: UDP livenessProbe: httpGet: path:
/api/system/lbstatus port: 9000 initialDelaySeconds: 180
readinessProbe: httpGet: path: /api/system/lbstatus port: 9000
initialDelaySeconds: 90
```

2.2.4 Prometheus и Grafana

Prometheus собирает метрики со всех сервисов через `/actuator/prometheus` endpoint. Grafana используется для визуализации.

Листинг 2.4 — Конфигурация Prometheus scrape targets

```
scrape_configs: - job_name: 'auth-service' static_configs: -
targets: ['auth-service:8081'] metrics_path: /actuator/prometheus
- job_name: 'task-service' static_configs: - targets: ['task-
service:8082'] metrics_path: /actuator/prometheus - job_name:
'notification-service' static_configs: - targets: ['notification-
service:8083'] metrics_path: /actuator/prometheus - job_name:
'postgres' static_configs: - targets: ['auth-postgres:9187',
'task-postgres:9187', 'notification-postgres:9187']
```

2.2.5 Jaeger

Jaeger развернут в режиме all-in-one для распределенной трассировки запросов между сервисами.

Листинг 2.5 — Конфигурация Jaeger Deployment

```
apiVersion: apps/v1 kind: Deployment metadata: name: jaeger
namespace: task-management spec: template: spec: containers: -
name: jaeger image: jaegertracing/all-in-one:1.51 env: - name:
COLLECTOR_OTLP_ENABLED value: "true" ports: - containerPort: 16686
name: query - containerPort: 6831 name: agent protocol: UDP -
containerPort: 14268 name: collector
```

2.3 Развертывание микросервисов

2.3.1 Auth Service

Auth Service реализован на Spring Boot с Kotlin. Обеспечивает регистрацию, авторизацию и хранение JWT токенов в Redis.

Листинг 2.6 — Helm values.yaml для Auth Service

```
replicaCount: 1 image: repository: auth-service tag: "latest"
service: type: ClusterIP port: 8081 resources: requests: cpu: 100m
memory: 256Mi limits: cpu: 300m memory: 512Mi autoscaling:
enabled: true minReplicas: 1 maxReplicas: 3
targetCPUUtilizationPercentage: 60 livenessProbe: httpGet: path:
/actuator/health/liveness port: 8081 initialDelaySeconds: 60
readinessProbe: httpGet: path: /actuator/health/readiness port:
8081 initialDelaySeconds: 40
```

2.3.2 Task Service

Task Service обеспечивает CRUD операции с задачами. При создании/обновлении задач публикует события в Kafka топик task-events.

Листинг 2.7 — Публикация событий в Kafka (KafkaProducerService.kt)

```
@Service class KafkaProducerService( private val kafkaTemplate:
KafkaTemplate<String, String>, private val objectMapper:
ObjectMapper ) { fun publishTaskEvent(eventType: String, task:
Task) { val event = TaskEvent( eventType = eventType, taskId =
task.id!!, title = task.title, userId = task.userId, timestamp =
Instant.now() ) kafkaTemplate.send("task-events",
objectMapper.writeValueAsString(event)) } }
```

2.3.3 Notification Service

Notification Service подписан на топики auth-events и task-events. При получении событий создает уведомления в базе данных.

Листинг 2.8 — Потребитель Kafka событий (KafkaConsumerService.kt)

```
@Service class KafkaConsumerService( private val
notificationService: NotificationService, private val
objectMapper: ObjectMapper ) { @KafkaListener(topics = ["auth-
events", "task-events"], groupId = "notification-service-group")
fun consume(message: String) { val event =
objectMapper.readValue(message, BaseEvent::class.java) val
notification = Notification( userId = event.userId, type =
event.eventType, message = generateMessage(event), read = false )
notificationService.save(notification) } }
```

2.3.4 KrakenD API Gateway

KrakenD выступает единой точкой входа для всех запросов. Настроена маршрутизация к сервисам и передача Authorization header.

Листинг 2.9 — Конфигурация KrakenD (krakend.json)

```
{ "$schema": "https://www.krakend.io/schema/v2.5/krakend.json",
"version": 3, "name": "Task Management API Gateway", "timeout":
"30000ms", "extra_config": { "security/cors": { "allow_origins":
["*"], "allow_methods": ["GET", "POST", "PUT", "DELETE",
"OPTIONS"], "allow_headers": ["*"] } }, "endpoints": [ {
"endpoint": "/auth/register", "method": "POST", "backend": [{
"url_pattern": "/api/auth/register", "host": ["http://auth-
service:8081"] }] }, { "endpoint": "/tasks", "method": "POST",
"input_headers": ["Authorization", "Content-Type"], "backend": [{
"url_pattern": "/api/tasks", "host": ["http://task-service:8082"]
}] } ] }
```

2.4 Конфигурация Helm Charts

Каждый сервис разворачивается через Helm Chart со следующими компонентами:

Листинг 2.10 — Структура Helm Chart

```
helm/service-name/ ├── Chart.yaml ├── values.yaml └── templates/
├── deployment.yaml ├── service.yaml ├── configmap.yaml ├──
secret.yaml ├── hpa.yaml └── servicemonitor.yaml
```

Листинг 2.11 — HorizontalPodAutoscaler (hpa.yaml)

```
apiVersion: autoscaling/v2 kind: HorizontalPodAutoscaler metadata:
name: {{ .Chart.Name }}-hpa spec: scaleTargetRef: apiVersion:
apps/v1 kind: Deployment name: {{ .Chart.Name }} minReplicas: {{
.Values.autoscaling.minReplicas }} maxReplicas: {{
.Values.autoscaling.maxReplicas }} metrics: - type: Resource
resource: name: cpu target: type: Utilization averageUtilization:
{{ .Values.autoscaling.targetCPUUtilizationPercentage }} - type:
Resource resource: name: memory target: type: Utilization
averageUtilization: 60
```

3 ПРОВЕРКА РАБОТОСПОСОБНОСТИ СИСТЕМЫ

3.1 Авторизация и работа с токенами

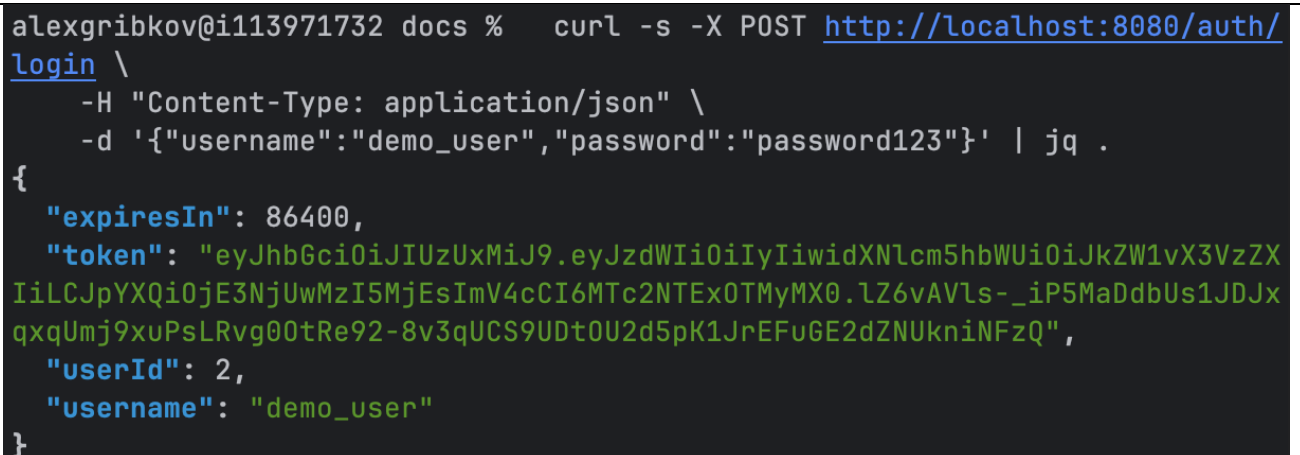
Регистрация нового пользователя и получение JWT токена:

Листинг 3.1 — Регистрация пользователя

```
curl -X POST http://localhost:8080/auth/register
-H "Content-Type: application/json"
-d
'{"username": "testuser", "email": "test@example.com", "password": "password123"}'
```

Листинг 3.2 — Авторизация и получение токена

```
curl -X POST http://localhost:8080/auth/login
-H "Content-Type: application/json"
-d '{"username": "testuser", "password": "password123"}'
```



```
alexgribov@i113971732 docs % curl -s -X POST http://localhost:8080/auth/login \
-H "Content-Type: application/json" \
-d '{"username": "demo_user", "password": "password123"}' | jq .
{
  "expiresIn": 86400,
  "token": "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiIyIiwidXNlcm5hbWUiOiJkZW1vX3VzZXIiLCJpYXQiOiJlZ3NjUwMzI5MjEsImV4cCI6MTc2NTExOTMyMX0.1Z6vAVls-_iP5MaDdbUs1JDJxqxqUmj9xuPsLRvg00tRe92-8v3qUCS9UDt0U2d5pK1JrEFuGE2dZNUkniNFzQ",
  "userId": 2,
  "username": "demo_user"
}
```

Рисунок 3.1 — Успешная авторизация и получение JWT токена

3.2 Создание и управление задачами

Создание задачи с использованием полученного JWT токена:

Листинг 3.3 — Создание задачи

```
curl -X POST http://localhost:8080/tasks
-H "Authorization: Bearer "
-H "Content-Type: application/json"
```

```
-d '{"title":"Test Task","description":"Test description","status":"PENDING"}'
alexgribov@i113971732 docs % curl -s -X POST http://localhost:8080/tasks \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"title":"Test Task","description":"Task for report screenshot","status":"PENDING"}' | jq .
{
  "createdAt": "2025-12-06T14:57:49.525313159Z",
  "description": "Task for report screenshot",
  "id": 2,
  "status": "PENDING",
  "title": "Test Task",
  "updatedAt": "2025-12-06T14:57:49.525317700Z",
  "userId": 2
}
```

Рисунок 3.2 — Создание задачи через API

3.3 Проверка Kafka

При создании задачи событие TASK_CREATED публикуется в топик task-events и потребляется Notification Service.

Листинг 3.4 — Просмотр топиков Kafka

```
kubectl exec -it kafka-0 -n task-management -
kafka-topics -list -bootstrap-server localhost:9092
```

```
alexgribkov@i113971732 docs % kubectl exec -it kafka-0 -n task-management -
- \
  kafka-topics --list --bootstrap-server localhost:9092

__consumer_offsets
auth-events
auth-events-dlt
auth-events-retry-1000
auth-events-retry-2000
task-events
task-events-dlt
task-events-retry-1000
task-events-retry-2000
alexgribkov@i113971732 docs % kubectl exec -it kafka-0 -n task-management -
- \
  kafka-console-consumer --bootstrap-server localhost:9092 \
  --topic task-events --from-beginning --max-messages 5
{"eventType":"TASK_CREATED","taskId":1,"userId":1,"title":"title","status":
"PENDING","timestamp":"2025-12-06T14:10:12.814292376Z"}
{"eventType":"TASK_CREATED","taskId":2,"userId":2,"title":"Test Task","stat
us":"PENDING","timestamp":"2025-12-06T14:57:49.564746643Z"}
```

Рисунок 3.3 — Kafka топики и события

3.4 Проверка Graylog

Все HTTP запросы логируются с информацией о методе, URL и IP-адресе отправителя.

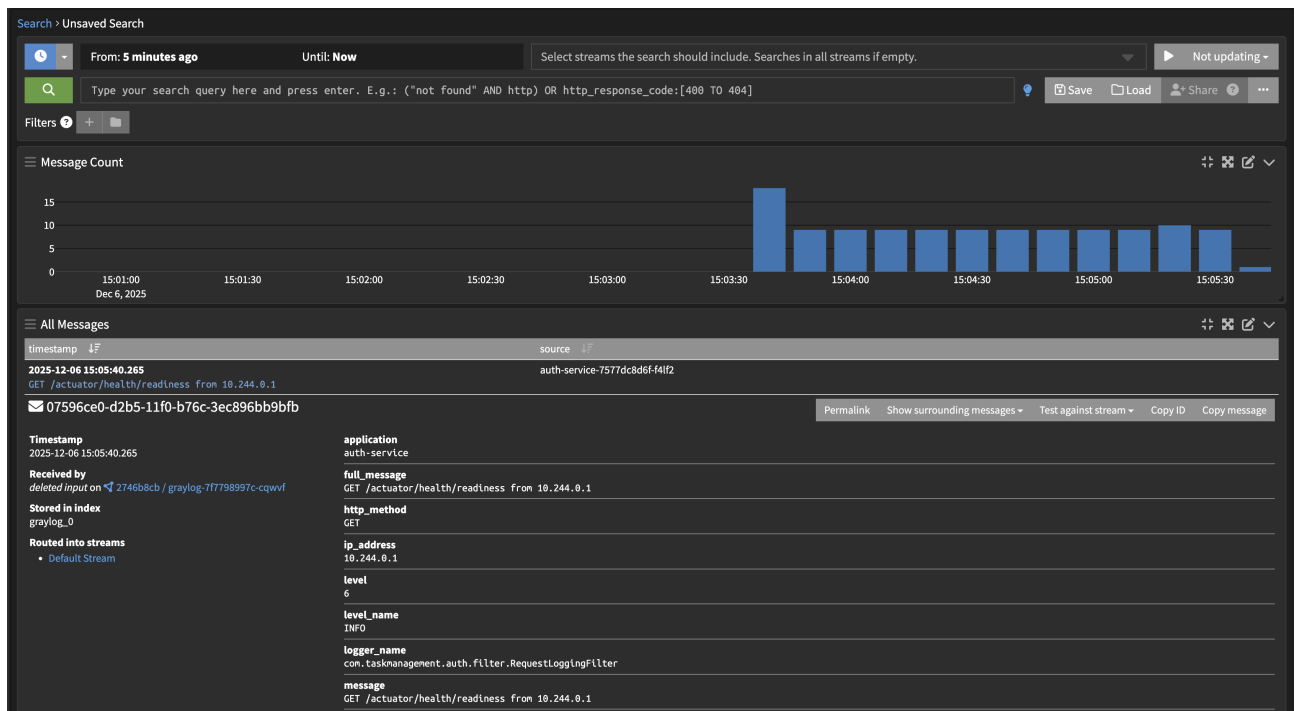


Рисунок 3.4 — Логи запросов в Graylog

3.5 Проверка Prometheus и Grafana

Prometheus собирает метрики со всех сервисов и баз данных. Grafana визуализирует данные.

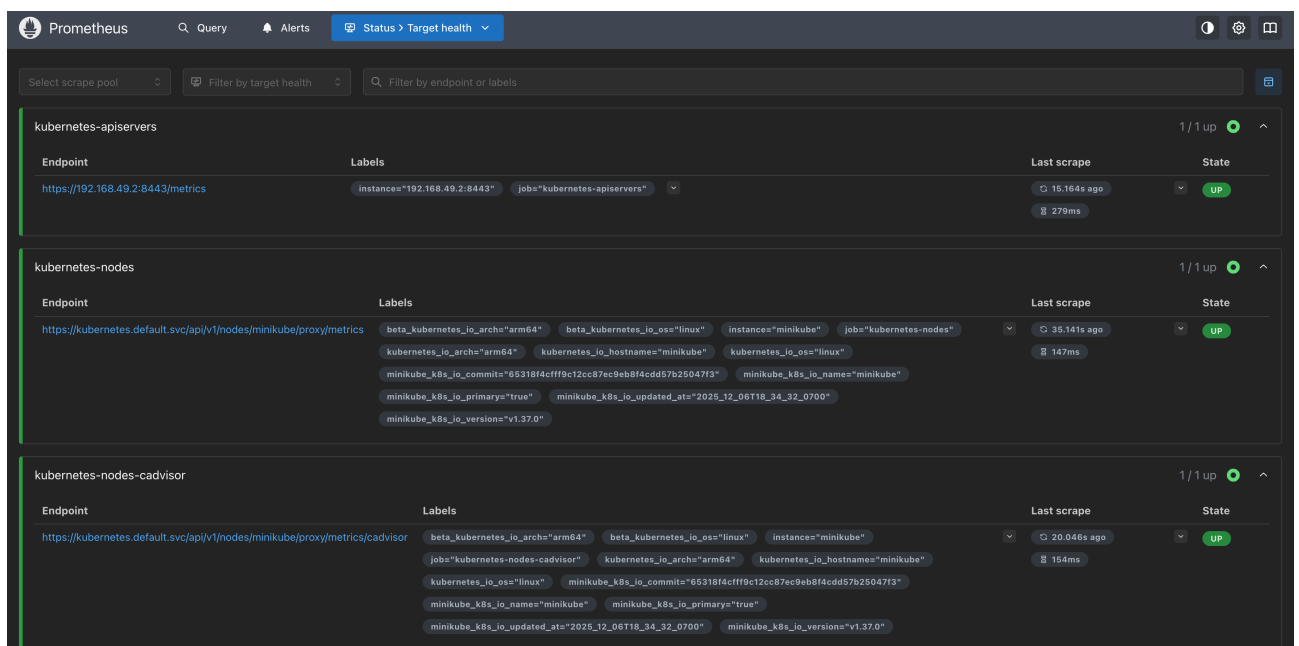


Рисунок 3.5 — Prometheus targets

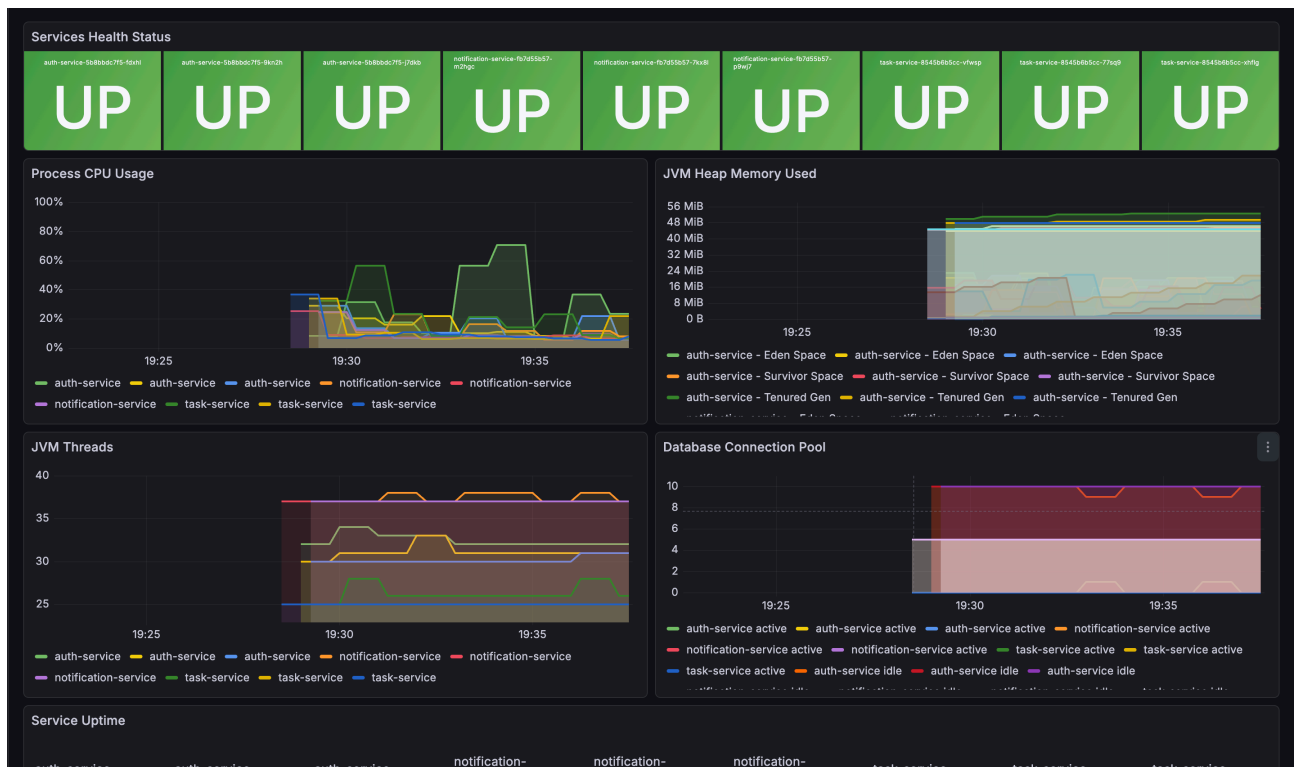


Рисунок 3.6 — Grafana dashboard с метриками сервисов

3.6 Проверка Jaeger

Jaeger отображает распределенные трейсы запросов через все сервисы.

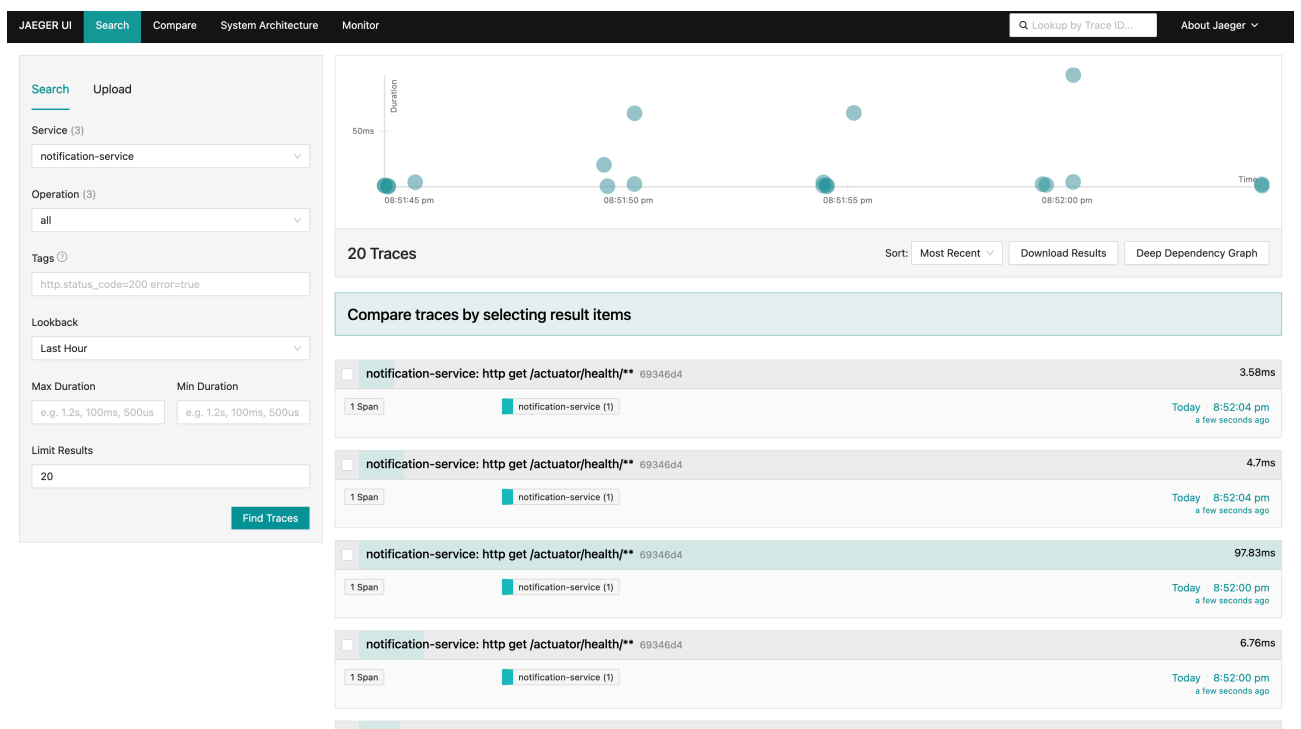


Рисунок 3.7 — Распределенный трейс запроса в Jaeger

3.7 Проверка Persistent Volume

Данные сохраняются при перезапуске пода с БД благодаря Persistent Volume.

Листинг 3.5 — Проверка сохранения данных

```
curl -X POST http://localhost:8080/auth/register
-H "Content-Type: application/json"
-d
'{"username":"persistent_user","email":"pv@test.com","password":"pass"}'
kubectl delete pod auth-postgres-postgresql-0 -n task-management
kubectl wait -for=condition=Ready pod/auth-postgres-postgresql-0
-n task-management -timeout=120s
curl -X POST http://localhost:8080/auth/login
-H "Content-Type: application/json"
-d '{"username":"persistent_user","password":"pass"}'
```



```

alexgribkov@i113971732 docs % curl -s -X POST http://localhost:8081/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"pv_test_user","password":"TestPass123"}' | jq .
{
  "token": "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiIyIiwidXNlcm5hbWUiOiJwdl90ZXN0X3VzZXIiLCJpYXQiOiJlE3NjUwNDQxNzIsImV4cCI6MTc2NTEzMDU3Mn0.L0jb8tVvFCD-fUow8_hDB9fyxx4rgubYZ-BTNCmUfFMtRXBw9VVSbv9FDdvfZ04TZoKqNVGNgZATXMA8i60Ew",
  "userId": 2,
  "username": "pv_test_user",
  "expiresIn": 86400
}
alexgribkov@i113971732 docs % kubectl delete pod auth-postgres-postgresql-0
-n task-management
pod "auth-postgres-postgresql-0" deleted from task-management namespace
alexgribkov@i113971732 docs % kubectl wait --for=condition=Ready pod/auth-postgres-postgresql-0 \
-n task-management
pod/auth-postgres-postgresql-0 condition met
alexgribkov@i113971732 docs % curl -s -X POST http://localhost:8081/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"pv_test_user","password":"TestPass123"}' | jq .
{
  "token": "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiIyIiwidXNlcm5hbWUiOiJwdl90ZXN0X3VzZXIiLCJpYXQiOiJlE3NjUwNDQzMjUsImV4cCI6MTc2NTEzMDcyNX0.q-8mBhuA8iQe1GK8aMoqfXgIW0W-ILsZr5ACbZtE0sSxTpbGcAXT0eboiEnd5e07Ttpwi82c3nDYqL_NFbZyqw",
  "userId": 2,
  "username": "pv_test_user",
  "expiresIn": 86400
}

```

Рисунок 3.8 — Сохранение данных после перезапуска пода с БД

4 ВЫВОД

В результате выполнения практической работы была разработана и развернута микросервисная система управления задачами со следующими результатами:

1. Реализованы 3 микросервиса на Spring Boot (Kotlin): Auth Service, Task Service, Notification Service
2. Настроена авторизация через JWT с хранением токенов в Redis
3. Реализовано асинхронное взаимодействие между сервисами через Kafka
4. Развернут KrakenD API Gateway для маршрутизации запросов
5. Настроено централизованное логирование через Graylog с HTTP методом, URL и IP адресом
6. Настроен мониторинг через Prometheus + Grafana с метриками сервисов и БД
7. Настроена распределенная трассировка через Jaeger
8. Обеспечено хранение данных в PostgreSQL с Persistent Volume (паттерн 1 БД на 1 сервис)
9. Настроены HPA для автомасштабирования при 60% CPU/Memory
10. Настроены Liveness и Readiness Probes для всех сервисов
11. Установлены лимиты ресурсов для всех компонентов

5 ОТВЕТЫ НА ВОПРОСЫ

1. Назовите плюсы и минусы использования Helm. Какие форматы конфигурационных файлов он поддерживает? Какие существуют аналоги?

Плюсы Helm:

- Шаблонизация манифестов с параметризацией через values.yaml
- Версионирование релизов и возможность отката
- Управление зависимостями между чартами
- Большой репозиторий готовых чартов

Минусы Helm:

- Сложность отладки шаблонов Go templates
- Необходимость изучения специфического синтаксиса
- Проблемы с управлением секретами

Helm поддерживает форматы YAML и JSON для конфигурационных файлов. Шаблоны используют Go templates.

Аналоги: Kustomize (встроен в kubectl), Jsonnet, Pulumi, Terraform (для IaC).

2. Почему для микросервисного взаимодействия используется шина сообщений? Какие аналоги существуют?

Преимущества шины сообщений:

- Асинхронность: сервисы не блокируются ожиданием ответа
- Слабая связанность: сервисы не знают друг о друге напрямую
- Отказоустойчивость: сообщения сохраняются при падении потребителя
- Масштабируемость: легко добавлять новых потребителей

Альтернативы межсервисного взаимодействия:

- REST/HTTP — простота, но синхронность и сильная связанность
- gRPC — высокая производительность, типизация, но требует компиляции proto-файлов
- GraphQL — гибкость запросов, но сложность реализации

Аналоги Kafka: RabbitMQ (AMQP), Apache Pulsar, NATS, Redis Streams.

3. Для чего используется Persistent Volume в БД? Преимущества и недостатки паттерна “1 БД на 1 сервис”?

Persistent Volume обеспечивает сохранение данных при перезапуске или удалении пода. Без PV данные теряются при каждом рестарте контейнера.

Преимущества паттерна “1 БД на 1 сервис”:

- Изоляция данных и независимость сервисов
- Возможность выбора оптимальной СУБД для каждого сервиса
- Независимое масштабирование
- Отсутствие single point of failure

Недостатки:

- Сложность обеспечения согласованности данных между сервисами
- Повышенное потребление ресурсов
- Сложность выполнения запросов, затрагивающих данные нескольких сервисов

4. Почему для мониторинга БД создается отдельный пользователь?

Отдельный пользователь для мониторинга создается по принципу минимальных привилегий (least privilege). Этому пользователю предоставляется только право на чтение системных метрик (SELECT на pg_stat_* представления).

Риски использования аккаунта с лишними правами:

- Компрометация системы мониторинга может привести к утечке или изменению данных
- Ошибка в конфигурации мониторинга может вызвать случайное изменение данных
- Нарушение требований аудита и compliance

5. Какой функционал предоставляет KrakenD? Какие существуют аналоги?

Функционал KrakenD:

- Маршрутизация запросов к backend сервисам
- Агрегация ответов от нескольких сервисов

- Rate limiting и throttling
- Аутентификация и авторизация (JWT validation)
- Кэширование ответов
- Circuit breaker
- Трансформация запросов и ответов
- Сбор метрик и трассировка

Аналоги: Kong, NGINX (с OpenResty), Traefik, Ambassador, Envoy, AWS API Gateway, Azure API Management.

6 СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Kubernetes: лучшие практики. — СПб.: Питер, 2021. — 288 с.: ил. — (Серия «Для профессионалов»).
2. KrakenD Documentation — Текст: электронный [сайт]. — URL: <https://www.krakend.io/docs/> (дата обращения: 06.12.2025).
3. Helm Documentation — Текст: электронный [сайт]. — URL: <https://helm.sh/docs/> (дата обращения: 06.12.2025).
4. Apache Kafka Documentation — Текст: электронный [сайт]. — URL: <https://kafka.apache.org/documentation/> (дата обращения: 06.12.2025).
5. Spring Boot Documentation — Текст: электронный [сайт]. — URL: <https://docs.spring.io/spring-boot/docs/current/reference/html/> (дата обращения: 06.12.2025).
6. Graylog Documentation — Текст: электронный [сайт]. — URL: <https://go2docs.graylog.org/5-2/home.htm> (дата обращения: 06.12.2025).